

Technical Design Document (TDD)

My Hotel Spy

Versão: 0.1

Engine sugerida: Unity 6 (URP)

Plataforma: WebGL (MVP)

Arquitetura: Data Driven + Component Based + Event Driven

1. Objetivo

O objetivo desta arquitetura é permitir que praticamente todo o conteúdo do jogo seja criado através de dados.

O código deve conhecer apenas regras.

Os mapas conhecem apenas posições.

Os objetos conhecem apenas seus estados possíveis.

O gerador monta toda investigação.

2. Arquitetura Geral

```
Game
|
|— Managers
|
|— Player
|
|— Case System
|
|— Object System
|
|— Evidence System
|
|— Report System
|
|— UI
```

|
└─ Data

Todo o fluxo gira em torno de um **Case Instance**.

3. Fluxo do Jogo

Game Start

↓

Seleciona Caso

↓

Case Generator

↓

Instancia mapa

↓

Instancia objetos

↓

Aplica estados

↓

Jogador investiga

↓

Jogador gera laudo

↓

Sistema compara

↓

Resultado

↓

4. Estrutura de Pastas

```
Assets/  
  
Art/  
  
Audio/  
  
Materials/  
  
Models/  
  
Prefabs/  
  
Player/  
  
Scripts/  
  
Data/  
  
UI/  
  
Scenes/  
  
Settings/
```

Dentro de Scripts:

```
Scripts/  
  
Core/  
  
Player/  
  
Interaction/  
  
Inspection/  
  
Objects/  
  
Evidence/  
  
Case/
```

Report/

Managers/

UI/

Utilities/

5. Core Managers

GameManager

Responsável por:

- Estado do jogo
- Carregar cena
- Fluxo principal

CaseManager

Responsável por:

- Criar investigação
- Pedir geração ao Case Generator
- Registrar objetos ativos

UIManager

Controla:

- HUD
- Briefing
- Tablet
- Relatório
- Menus

SaveManager

Responsável por:

- progresso

- dinheiro
- cosméticos

AudioManager

Centraliza todos os efeitos sonoros.

6. Sistema de Estados do Jogo

Main Menu

↓

Briefing

↓

Loading

↓

Inspection

↓

Report

↓

Results

↓

Next Case

Cada estado controla apenas sua responsabilidade.

7. Player

Componentes:

Player

Character Controller

Camera

Interaction

Tool Controller

Inspection Controller

8. Sistema de Ferramentas

Classe base:

Tool

Implementações:

ZoomTool

FlashlightTool

UVTool

RFSscannerTool

ThermalTool

EndoscopeTool

Todas herdam da mesma interface.

ITool

Métodos:

Activate()

Deactivate()

Inspect()

Update()

CanInspect()

9. Sistema de Interação

Todo objeto interativo implementa:

IInteractable

Métodos:

Interact()

Inspect()

Highlight()

GetInspectionData()

Assim qualquer objeto pode ser inspecionado.

10. Sistema de Objetos

Classe principal

InspectableObject

Possui:

ID

Object Type

Current State

Evidence List

Mesh

Collider

Interaction Point

O objeto nunca sabe qual caso está acontecendo.

Ele apenas recebe um estado.

11. Biblioteca de Objetos

Cada objeto possui uma definição.

Exemplo:

Smoke Detector

Possible States

Normal

Camera

Microphone

Old Model

Outro:

Clock

Normal

Camera

Microphone

Secret Compartment

Essas definições ficam em Data.

12. Estados

Classe:

ObjectState

Contém:

Nome

Categoria

Irregularidade

Evidências possíveis

Modelo opcional

Material opcional

Peso para geração

13. Sistema de Evidências

Classe

Evidence

Campos:

ID

Nome

Ferramenta

Descrição

Peso

Confiança

Exemplo:

Lens Reflection

Ferramenta

Flashlight

Confiança

0.4

O jogador acumula evidências.

Nenhuma evidência prova sozinha uma irregularidade.

14. Sistema de Casos

Classe principal

CaseDefinition

Contém:

ID

Nome

Mapa

Dificuldade

Categoria

Pool de denúncias

Pool de objetos

Número mínimo

Número máximo

Quando iniciado:

Generate()

Produz

15. Case Instance

Representa uma investigação única.

Contém:

Seed

Mapa

Objetos modificados

Estados escolhidos

Denúncia

Data

Resultado esperado

É salvo até terminar a partida.

16. Sistema Procedural

O gerador funciona em etapas.

Passo 1

Escolher mapa.

↓

Passo 2

Escolher seed.

↓

Passo 3

Selecionar objetos elegíveis.

↓

Passo 4

Sortear estados.

↓

Passo 5

Gerar evidências.

↓

Passo 6

Gerar denúncia.

↓

Caso pronto.

17. Seed

Toda investigação utiliza uma seed.

Isso permite:

- repetir casos
- compartilhar casos
- depurar geração

Exemplo

Seed

8429183

Sempre gera a mesma investigação.

18. Sistema de Relatório

Classe

InspectionReport

Contém

Objetos marcados

Conclusões

Tempo

Precisão

Falsos positivos

Acertos

Erros

Nota

Comparado automaticamente com:

Case Instance

19. Sistema de Dados

Todos os dados devem ficar separados do código.

Exemplo:

Objects/

Clock.json

SmokeDetector.json

Mirror.json

Cases/

Apartment01.json

Hotel02.json

Evidence/

Reflection.json

RFSignal.json

Glue.json

20. Organização dos Prefabs

Cada objeto possui um prefab.

Clock

Prefab

↓

Mesh

↓

InspectableObject

↓

Interaction Point

Os estados alteram apenas:

- componentes
- materiais
- dados

Não criam novos prefabs.

21. Comunicação

Evitar referências diretas.

Utilizar eventos.

Exemplo

Object Inspected

↓

Evidence Found

↓

UI Updated

Ao invés de

Objeto chama UI diretamente.

22. Arquitetura dos Dados

Map

↓

Spawn Points

↓

Object Definition

↓

Object State

↓

Evidence

↓

Case

↓

Report t

Tudo deriva dessa cadeia.

23. MVP

Para validar a arquitetura basta implementar:

Player

↓

Interação

↓

1 mapa

↓

10 objetos

↓

3 estados

↓

2 ferramentas

↓

Case Generator

↓

Laudo

↓

Resultado

Nada além disso.

24. Princípios de Código

- Single Responsibility.
 - Componentes pequenos.
 - Dados separados do código.
 - Evitar heranças profundas.
 - Preferir composição.
 - Nunca codificar casos específicos.
 - Toda regra deve ser reutilizável.
 - Todo conteúdo deve vir de dados.
-

25. Convenções

Classes:

```
PascalCase
```

Métodos:

```
PascalCase
```

Variáveis privadas:

```
_camelCase
```

Interfaces:

```
IInteractable
```

```
ITool
```

Enums:

```
ToolType
```

```
EvidenceType
```

```
ObjectStateType
```

26. Extensibilidade

O projeto deve permitir adicionar novos conteúdos sem alterar sistemas existentes.

Exemplos:

Novo objeto

→ criar JSON + prefab.

Nova evidência

→ criar definição.

Novo mapa

→ adicionar pontos de interesse.

Novo caso

→ adicionar definição.

Novo tipo de irregularidade

→ registrar novo estado.

Nenhum desses cenários deve exigir mudanças no código do gerador.

27. Futuras Expansões (fora do MVP)

- Mais ferramentas.
 - Mais mapas.
 - Objetos inteligentes (IoT).
 - Compartimentos secretos avançados.
 - Casos especiais totalmente roteirizados.
 - Sistema de estatísticas do jogador.
 - Compartilhamento de seeds.
 - Editor interno de casos.
 - Workshop de mapas (caso o projeto migre para Steam).
-

28. Princípio Central da Arquitetura

A arquitetura deve seguir uma única regra:

O conteúdo do jogo não são os mapas. O conteúdo do jogo são os casos.

Os mapas apenas fornecem o ambiente.

Os objetos fornecem possibilidades.

Os estados fornecem irregularidades.

As evidências fornecem pistas.

O gerador cria uma investigação única a partir dessas peças.

Todo o restante do jogo existe para permitir ao jogador resolver essa investigação.